

Come Far Costruire il Tuo Software Senza Bruciare Soldi

Guida pratica per imprenditori che devono sviluppare un'applicazione su misura e non vogliono scoprire troppo tardi di aver sbagliato fornitore, requisiti, o entrambe le cose.

Il settanta per cento dei progetti software personalizzati sfora i tempi o non viene mai completato. Non perché lo sviluppo software sia impossibile — ma perché la maggior parte dei progetti parte male. Parte senza aver risposto alla domanda più importante: hai davvero bisogno di software su misura? E se sì, come ti assicuri che quello che costruisci sia quello che ti serve?

Questo manuale ti insegna a rispondere a queste domande, a scegliere il fornitore giusto, a gestire il progetto come cliente non tecnico, e a non scoprire a consegna avvenuta che il software non funziona come ti aspettavi.

1. La domanda che quasi nessuno si fa prima di iniziare

Hai bisogno di un software su misura. Non un template, non un plugin WordPress. Qualcosa che faccia esattamente quello che serve alla tua azienda.

Prima di parlare con qualsiasi fornitore, fai questa domanda: c'è già qualcosa sul mercato che risolve il mio problema, anche parzialmente?

La risposta non è scontata. Gli imprenditori che arrivano a chiedere sviluppo custom spesso non l'hanno mai cercata davvero. Hanno un problema specifico, hanno sentito che il software su misura è la risposta, e si sono fermati lì.

Quando ha senso il custom:

- Nessun software commerciale copre il tuo processo specifico, e hai già verificato attivamente le alternative
- Il volume di lavoro giustifica l'investimento, e puoi calcolare il ritorno in modo concreto
- Hai un vantaggio competitivo che dipende da come lavori — non da quale strumento usi, ma da un processo che non puoi comprare altrove
- Devi integrare sistemi esistenti che non parlano tra loro, e le integrazioni native non bastano

Quando probabilmente non serve il custom:

- "Voglio qualcosa come X ma con una funzione diversa" — prova prima X, adattati al 90% della soluzione che esiste
- Il processo che vuoi automatizzare cambia ogni mese — prima stabilizzalo, poi digitalizzalo
- Non hai ancora chiaro cosa deve fare il software — prima fai consulenza, poi sviluppo. Un fornitore che ti fa sviluppare senza chiarezza ti prende i soldi e ti consegna qualcosa di sbagliato
- Strumenti no-code o low-code potrebbero risolvere il problema — molte automazioni di processo che le aziende credono richiedano sviluppo possono essere realizzate in giorni, senza scrivere codice

Il test del ritorno.

Chiediti: se questo software funzionasse perfettamente, quanto risparmio o guadagno nei prossimi dodici mesi? Se la risposta è inferiore a tre volte il costo di sviluppo stimato, probabilmente non conviene. Se è superiore a cinque volte, vale la pena esplorare. Questo non è un calcolo preciso — è un filtro per evitare decisioni emotive.

***Insight 108 Vision** — Con vent'anni di architettura enterprise alle spalle, ho visto molte aziende spendere risorse significative in sviluppo custom per risolvere problemi che uno strumento esistente avrebbe risolto in giorni. La domanda "esiste già qualcosa che funziona?" non è una resa — è il primo atto di intelligenza imprenditoriale.*

2. Capire le opzioni: chi può costruire il tuo software

Prima di valutare i fornitori, è utile capire le categorie. Ogni tipo di fornitore ha profili di rischio e vantaggi diversi. Non esiste la scelta giusta in assoluto — esiste quella giusta per il tuo progetto, la tua complessità, il tuo modo di lavorare.

Opzione | Punti di forza | Rischi reali | Adatta per

Freelancer | Rapporto diretto, costi contenuti, flessibilità | Rischio abbandono a progetto in corso, nessuna governance, nessun peer review sul c

Software house | Team strutturato, capacity garantita, processi definiti | Costo alto, ti assegnano spesso chi è disponibile invece del senior che

Boutique tecnica | Qualità tecnica alta, relazione diretta, attenzione al contesto | Capacity limitata, possibili colli di bottiglia su progetti g

Factory / team in retainer | Continuità, conoscenza del sistema nel tempo, evoluzione costante | Richiede impegno continuativo, non adatta a svilu

Team interno | Controllo totale, conoscenza profonda del business | Costo fisso alto, difficile recruiting, rischio concentrazione know-how | Pr

3. Come riconoscere il fornitore sbagliato (e quello giusto)

Questa è la parte del manuale che ha più valore pratico. Ho visto decine di progetti fallire, e in quasi tutti i casi i segnali erano visibili prima che il contratto fosse firmato.

Segnali di allarme — fermati prima di firmare

Ti dà un prezzo senza aver capito cosa serve.

"Sì sì, facciamo tutto, costano X." Se dopo dieci minuti di conversazione hai già un preventivo, non hai parlato con qualcuno che vuole fare un buon lavoro — hai parlato con qualcuno che vuole chiudere una vendita. La stima reale di un progetto richiede domande, approfondimento, e tempo.

Non fa domande sul tuo business.

Un buon fornitore tecnico dovrebbe capire cosa fai, perché lo fai, e chi usa il software prima di scrivere una riga di codice. Se non è curioso del tuo business, non costruirà qualcosa di utile per il tuo business.

Non parla di test, deploy, e documentazione.

Lo sviluppo è la parte visibile. I test, il deployment automatizzato, e la documentazione sono la parte che determina se il software funziona in produzione e se qualcuno riuscirà a mantenerlo tra due anni. Se il fornitore non li menziona, sono costi nascosti che pagherai dopo.

Non ti mostra lavori precedenti simili.

Qualsiasi fornitore serio ha un portfolio. Se non può mostrarti niente di simile a quello che ti serve — o se non ti permette di parlare con clienti esistenti — il rischio che il tuo progetto diventi il banco di prova per qualcosa che non ha mai fatto prima è reale.

Vuole l'intero importo anticipato.

Non esiste un motivo valido per pagare tutto prima che venga consegnato qualcosa di tangibile. Un fornitore che chiede il cento per cento anticipato sta scaricando su di te tutto il rischio finanziario del progetto.

Non propone milestone intermedie.

Un progetto senza milestone è un progetto dove scopri i problemi solo alla consegna — quando è tardi e costoso

correggere.

Segnali positivi — buon indicatore di serietà

Ti fa più domande di quante ne fai tu.

Vuol dire che sta cercando di capire davvero cosa serve, non solo di chiudere il contratto.

Propone una fase di Discovery pagata prima di darti il prezzo.

Ci arriviamo nella prossima sezione. È il segnale più forte di professionalità.

Parla di architettura, non solo di funzionalità.

Le funzionalità descrivono cosa fa il software oggi. L'architettura determina se potrà evolvere domani senza crollare.

Definisce chiaramente cosa è dentro e cosa è fuori dallo scope.

Un buon fornitore sa che la chiarezza sullo scope protegge entrambe le parti. Se è vago su questo, i problemi emergeranno dopo.

Ti propone pagamenti a milestone.

Significa che si fida del proprio lavoro abbastanza da ricevere il pagamento solo quando consegna qualcosa di verificabile.

Ti dà accesso al codice dal giorno uno.

Il codice è tuo. Un fornitore che lo tiene sotto controllo esclusivo sta costruendo una dipendenza. Il tuo codice deve essere accessibile a te dal primo commit.

4. La Discovery: la fase che non puoi saltare

La Discovery è la fase in cui il fornitore — prima di scrivere una riga di codice — capisce cosa serve, progetta l'architettura di base, e ti fornisce una stima reale.

Dura tipicamente una o due settimane. È pagata. Ed è la differenza più importante tra un progetto che parte bene e uno che parte male.

Perché pagare la Discovery

La Discovery gratuita ha un incentivo sbagliato: il fornitore la fa in fretta perché vuole arrivare al contratto, non perché vuole produrre qualcosa di utile. Il risultato è un documento superficiale scritto per giustificare una stima, non per chiarire il progetto.

La Discovery pagata ha l'incentivo giusto: il fornitore riceve una compensazione per un lavoro specifico — produrre un documento che vale indipendentemente da quello che succede dopo. Se alla fine della Discovery decidi di non procedere, hai comunque in mano qualcosa di utile. Se decidi di procedere con un altro fornitore, puoi usare quel documento come base.

Un fornitore che non propone una Discovery — o la fa gratis in un'ora — probabilmente non sa cosa sta costruendo. O lo sa, ma non vuole che tu lo sappia.

Cosa devi ricevere alla fine della Discovery

Un documento scritto in italiano comprensibile che contiene:

- Specifiche funzionali: cosa fa il software, scritto in modo che tu possa verificarlo senza essere tecnico. "L'utente può caricare un file PDF e vedere il risultato elaborato entro trenta secondi" è una specifica funzionale. "Implementare il modulo di upload" non lo è.
- Architettura di base: un diagramma con spiegazione — come sono organizzati i componenti principali, dove vivono i dati, come comunicano le parti.
- Scope chiaro: lista esplicita di cosa è incluso e cosa non è incluso. Questa lista è il documento contrattuale più importante che firmerai.
- Timeline per fasi: non una data di consegna unica. Fasi intermedie con deliverable verificabili.
- Rischi identificati: cosa potrebbe rallentare o complicare il progetto, con indicazione di come gestirli. Un fornitore che non identifica rischi non è ottimista — non ha guardato a fondo.

5. Come scrivere requisiti che funzionano

La maggior parte dei progetti software fallisce non perché il fornitore sia incompetente, ma perché i requisiti erano ambigui. Il fornitore ha costruito quello che ha capito. Tu volevi qualcosa di diverso. Nessuno ha torto — ma il costo per correggere è tuo.

Il formato che funziona: User Story

Le User Story descrivono il comportamento del software dal punto di vista di chi lo usa, non dal punto di vista tecnico.

Il formato base è: "Come [chi sono], voglio [cosa voglio fare], in modo da [perché mi serve]."

Esempio sbagliato: "Implementare un sistema di notifiche email."

Esempio corretto: "Come responsabile commerciale, voglio ricevere un'email quando un'offerta che ho inviato viene aperta dal cliente, in modo da poterlo contattare nel momento più opportuno."

La differenza non è estetica. La User Story corretta contiene informazioni che il tuo tecnico non potrebbe ricavare dall'altra versione: chi la usa, in quale contesto, e quale problema risolve. Ogni informazione che manca dalla specifica è una decisione che prenderà il fornitore al posto tuo.

Criteri di accettazione

Per ogni requisito importante, scrivi come verificherai che sia fatto correttamente. Non "il sistema invia email" — ma "dopo che l'offerta viene aperta, il responsabile riceve l'email entro cinque minuti, con il nome del cliente e il titolo dell'offerta nel testo."

I criteri di accettazione sono il tuo strumento di verifica. Senza di essi, "fatto" può significare cose diverse per te e per il fornitore.

Come raccogliere i requisiti con il tuo team

Le persone che useranno il software ogni giorno fanno cose che tu non sai. Non coinvolgerle nella raccolta dei requisiti è uno degli errori più costosi che un committente non tecnico possa fare.

Il metodo più semplice: fai una sessione di un'ora con le persone che useranno il software. Chiedi una cosa sola: "Nell'attività che fai ogni giorno, qual è la cosa più frustrante, quella che ti fa perdere più tempo, quella che ti sembra più assurda?" Le risposte sono i tuoi requisiti.

****Insight 108 Vision**** — Un requisito ambiguo è un contratto di litigio. Non è pessimismo — è la meccanica di come funziona lo sviluppo. Più tempo investi a chiarire prima, meno tempo e denaro spenderai a correggere dopo. Il rapporto è tipicamente uno a dieci: un'ora di chiarimento prima vale dieci ore di correzione dopo.

6. Gestire il progetto come cliente non tecnico

Non devi diventare un esperto tecnico per gestire bene un progetto software. Devi seguire poche regole che proteggono i tuoi interessi e mantengono il progetto sul binario giusto.

Regola 1 — Demo ogni due settimane, senza eccezioni

Non accettare mai "ci vediamo tra due mesi con il prodotto finito." Ogni due settimane devi vedere qualcosa che funziona — anche parziale, anche incompleto, ma funzionante.

Perché è fondamentale: se aspetti due mesi per vedere il risultato, e il risultato non è quello che volevi, hai due mesi di sviluppo da correggere. Se vedi una demo ogni due settimane e qualcosa non va, hai due settimane di lavoro da correggere. La differenza in termini di costo è enorme.

Una demo non è una presentazione di slide. È il software vivo, che puoi toccare, testare, e confrontare con quello che avevi in mente. Se il fornitore non riesce a mostrarti qualcosa di funzionante ogni due settimane, c'è un problema — di processo, di competenza, o entrambi.

Regola 2 — Tutto quello che non è scritto non è incluso

Lo scope del progetto è il documento firmato da entrambe le parti. Tutto quello che il software deve fare è scritto lì. Se non è scritto, non è incluso nel prezzo concordato.

Quando nel corso del progetto ti vengono nuove idee — e ti verranno, è normale — il processo corretto è un change request: il fornitore valuta l'impatto, ti dà una stima, tu decidi se procedere. Non è un ostacolo — è trasparenza. Un fornitore che aggiunge cose senza aggiornare il contratto non è generoso, sta accumulando debito da riscuotere più avanti.

Regola 3 — Struttura i pagamenti a milestone

Una struttura sana distribuisce il rischio tra te e il fornitore:

- Una parte alla firma — il fornitore ha bisogno di un'iniezione di liquidità per partire, e tu dimostri serietà
- Una parte a metà progetto — quando vedi la prima demo sostanziale e puoi verificare che il lavoro stia procedendo nella direzione giusta
- La parte finale alla consegna — quando accetti il risultato come conforme ai criteri di accettazione

Questa struttura allinea gli incentivi: il fornitore è motivato a consegnare qualcosa di buono perché una parte del suo compenso dipende dalla tua accettazione.

Regola 4 — Il codice è tuo, dal giorno uno

Dal primo giorno di sviluppo devi avere accesso al repository dove vive il codice — su GitHub, GitLab, o qualsiasi altro sistema di controllo versione. Non è una questione tecnica: è una questione di proprietà.

Se il fornitore tiene il codice sotto controllo esclusivo, hai creato una dipendenza totale. Se sparisce, se litigate, se trovate un accordo economicamente insostenibile — tu non hai nulla. Il tuo codice deve essere accessibile a te sempre.

Regola 5 — La documentazione non è un extra

La documentazione è parte del prodotto. Non è qualcosa che si aggiunge dopo, a progetto finito, come omaggio. È il manuale di istruzioni di quello che è stato costruito.

Senza documentazione, tra sei mesi — che tu sia con lo stesso fornitore o con un altro — nessuno capirà come funziona il sistema. Ogni modifica sarà più rischiosa, ogni nuovo sviluppatore impiegherà settimane per orientarsi, ogni

problema sarà più difficile da diagnosticare.

Includi la documentazione esplicitamente nei criteri di accettazione, con un formato minimo definito: struttura del sistema, istruzioni di deployment, descrizione delle scelte architetturali principali.

7. Cosa deve avere un software costruito bene

Non devi essere tecnico per verificare queste cose. Devi solo saperle chiedere — e valutare la risposta.

Elemento | Cosa chiedere | Perché conta

Test automatici | "Ci sono test nel codice? Quale percentuale del codice è coperta da test?" | Se non ci sono test, ogni modifica rischia di rompere

Deploy automatizzato | "Come va il software in produzione?" | Se la risposta è "carichiamo i file manualmente", hai un processo rischioso, non rip

Monitoraggio | "Come sapete se qualcosa non funziona?" | Se la risposta è "ce lo dicono gli utenti", il tuo sistema non ha allerta preventiva. I p

Backup dei dati | "I dati degli utenti sono backuppati? Con quale frequenza? Dove?" | Se non c'è una risposta chiara e documentata, c'è un rischio

Sicurezza | "Avete fatto una revisione di sicurezza? Come vengono gestite le password degli utenti?" | Se la risposta è vaga o sorpresa, stai cost

Queste domande non richiedono competenza tecnica per essere poste. Richiedono solo di pretendere risposte chiare. Un fornitore che risponde chiaramente a tutte e cinque è un fornitore che sa cosa sta facendo. Uno che diventa difensivo o vago su una o più di queste ha qualcosa da nascondere.

*****Insight 108 Vision**** — Il mio team ha misurato che le aziende che adottano test automatici e deployment automatizzato fin dall'inizio riducono il tasso di bug in produzione del novantotto per cento rispetto a quelle che non lo fanno. Non è un numero teorico — è il risultato misurabile di un approccio diverso alla qualità.*

8. Le fasi del progetto: dalla Discovery al go-live

Un progetto ben strutturato non è una linea retta da "hai l'idea" a "hai il software". È una sequenza di fasi con deliverable verificabili. Conoscere questa sequenza ti permette di sapere in ogni momento a che punto siete, cosa aspettarti dopo, e quando allarmarsi.

Fase 0 — Discovery (una-due settimane)

Già descritta in dettaglio. Output: documento di specifiche, architettura di base, scope firmato, stima per le fasi successive.

Fase 1 — Setup e fundamenta (una-due settimane)

Il team configura l'ambiente di sviluppo, i sistemi di controllo versione, la pipeline di deployment, e l'ambiente di test. Alla fine di questa fase non c'è ancora nulla di "visibile" per te, ma c'è tutta l'infrastruttura che permette di lavorare in modo sicuro e tracciabile.

Fase 2 — Sviluppo iterativo (durata variabile, sprint di due settimane)

Il cuore del progetto. Ogni sprint produce funzionalità testabili, con demo al termine. Tu vedi il lavoro, fornisci feedback, il team integra le correzioni nel prossimo sprint. Non aspettare l'ultimo sprint per fare domande o sollevare problemi — il

costo di correggere cresce ad ogni settimana di ritardo.

Fase 3 — Test e stabilizzazione (due-quattro settimane)

Prima del go-live, una fase dedicata a trovare e correggere i problemi. Test funzionali (fa quello che deve fare?), test di performance (regge sotto carico?), test di sicurezza (base). Questa fase non va compressa o saltata per rispettare una deadline.

Fase 4 — Go-live e post-lancio (prime due-quattro settimane in produzione)

Il lancio non è la fine del progetto. È l'inizio di una fase diversa. I problemi che emergono nelle prime settimane di uso reale sono normali — l'importante è che ci sia un fornitore disponibile a risolverli rapidamente, e che tu sappia chi chiamare e come.

9. Dopo il go-live: la manutenzione non è un'opzione

Il software non è "finito" il giorno della consegna. È vivo. E tutto ciò che è vivo richiede cura.

La garanzia post-lancio copre i bug emersi dopo il rilascio — comportamenti non conformi alle specifiche concordate. Di solito dura trenta-novanta giorni. Non copre nuove funzionalità, cambiamenti di idea, o problemi causati da modifiche fatte da terze parti.

La manutenzione evolutiva è tutto ciò che viene dopo: aggiornamenti di sicurezza dei componenti usati, adattamento a nuovi sistemi operativi o browser, piccole migliorie richieste dagli utenti, monitoraggio del funzionamento. Un software che non riceve manutenzione diventa progressivamente obsoleto e vulnerabile.

Il rischio del fornitore sparito. Se costruisci il tuo software con qualcuno e poi il rapporto si interrompe — per qualsiasi motivo — devi poter continuare. Questo richiede: codice tuo e accessibile, documentazione tecnica adeguata, e architettura che non sia incomprensibile senza l'autore originale.

La forma di relazione continuativa più efficace per la maggior parte delle PMI è il modello factory o retainer: un fornitore che conosce già il tuo sistema e lo fa evolvere ogni mese, con un impegno di capacity prevedibile da entrambe le parti.

***Insight 108 Vision** — Ho visto aziende bloccarsi per mesi perché il loro fornitore storico era irreperibile e il codice era incomprensibile senza di lui. Il lock-in tecnologico non è un problema teorico — è un rischio operativo concreto che si previene con clausole semplici e buone pratiche dal primo giorno.*

10. Le domande da fare prima di firmare

Prima di firmare qualsiasi contratto con un fornitore di sviluppo software, ottieni risposte chiare a queste domande. Non come parte di una presentazione — in forma scritta, nel contratto o in un allegato.

Checklist di due diligence pre-contratto:

- ☐ Chi sono le persone specifiche che lavoreranno al mio progetto? (nome, esperienza, disponibilità)
- ☐ Ho visto almeno tre lavori precedenti simili per complessità o settore?
- ☐ Ho parlato con almeno un cliente esistente del fornitore?
- ☐ Il contratto specifica cosa succede se i tempi slittano (penali, obblighi, procedure)?
- ☐ Ho accesso al repository dal primo giorno, senza condizioni?
- ☐ I pagamenti sono strutturati a milestone verificabili?

- [] Ci sono demo obbligatorie ogni due settimane, scritte nel contratto?
- [] La documentazione tecnica è un deliverable esplicito, non un'opzione?
- [] Ho una clausola di uscita chiara se il rapporto non funziona?
- [] Lo scope è scritto e firmato da entrambe le parti, con lista esplicita di esclusioni?

Un fornitore professionale non si sorprende di queste domande. Le rispetta, perché proteggono anche lui da ambiguità future.

Vuoi andare oltre?

Vuoi applicare questo metodo alla tua azienda? Prenota 30 minuti con noi su 108vision.it — gratuito, senza impegno.

108 Vision — Costruiamo la direzione, non solo il codice.